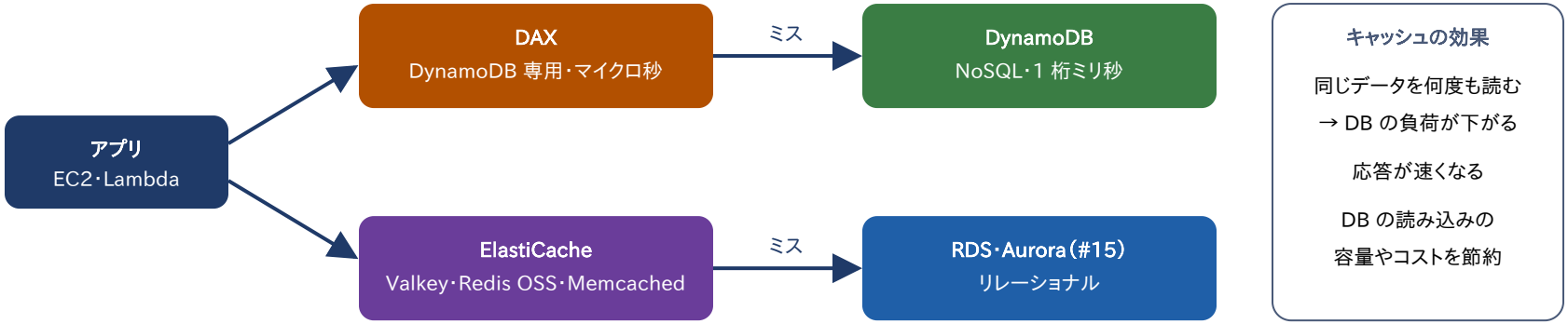
AWS SAA（SAA-C03）　分野3 高パフォーマンス　#16

DynamoDB とキャッシュ

サーバーレスの NoSQL データベース DynamoDB のキーと容量、グローバルテーブルやバックアップなどの機能、DAX と ElastiCache によるキャッシュ

各ページの下に、根拠にした AWS の公式ドキュメント（Amazon DynamoDB デベロッパーガイド・Amazon ElastiCache ユーザーガイド）を書いた（青い文字を押すと開く）。「」の中はドキュメントの原文どおり（日本語版は機械翻訳のため、語順が不自然なところもそのまま）。テーブル名・数値・リージョンは Claude が作った例。数字は 2026-11 時点のドキュメントによる

全体像：DB の前と横 試験ガイド タスク 3.3



DynamoDB の見分け方：サーバーレスの NoSQL。キーと値・ドキュメント。JOIN なし。どんな規模でも 1 桁ミリ秒。問題文に「サーバーレス」「キーと値」「数百万リクエスト」「スキーマが柔軟」が出たら候補。

キャッシュの選び方：DynamoDB の前ならコードの変更が少ない DAX。RDS などほかの DB の前や、セッション・順位表なら ElastiCache。CloudFront のキャッシュは #19。

出典：Amazon DynamoDB とは（「Amazon DynamoDB は、サーバーレスのフルマネージド分散 NoSQL データベースで、あらゆる規模で 1 桁ミリ秒のパフォーマンスを実現します。」「リレーショナルデータベースとは異なり、DynamoDB は JOIN 演算子をサポートしません。」）／Amazon ElastiCache とは（「Amazon ElastiCache は Valkey、Memcached、および Redis OSS のエンジンで動作します。」）

第1章

DynamoDB のしくみと容量

パーティションキーとソートキー、GSI と LSI、オンデマンドとプロビジョンド、RCU と WCU の計算、結果整合性と強力な整合性、トランザクション、テーブルクラス。

テーブル・キー・インデックス

テーブル GameScores

UserId（PK）	GameTitle（SK）	TopScore
101	Galaxy Invaders	5842
101	Meteor Blasters	1000
102	Meteor Blasters	2400

UserId で Query は速い。GameTitle だけでは Scan になる

GSI：PK=GameTitle、SK=TopScore
違うパーティションキーで引ける・あとから作れる
結果整合性の読み込みだけ・既定で 20 個まで

LSI：PK=UserId のまま、SK=別の属性
同じパーティションの中で別の順に・作成時だけ
強力な整合性の読み込みも使える

キーの決め方：パーティションキーで置き場所が決まり、ソートキーで同じパーティションの中の順が決まる。項目は 1 つ 400 KB まで。JOIN がないので、よく使う問い合わせの形に合わせてキーとインデックスを設計する。

GSI と LSI のひっかけ：「運用中のテーブルに新しい検索条件」→ GSI（LSI はあとから足せない）。「同じキーで別の並び順、強い整合性も」→ 作成時に LSI。

出典：Amazon DynamoDB のコアコンポーネント（「ローカルセカンダリインデックス – パーティションキーはテーブルと同じですが、ソートキーが異なるインデックスです。」）／ローカルセカンダリインデックス（「テーブルのローカルセカンダリインデックスは、テーブルが作成された時点で作成されます。」）／DynamoDB のグローバルセカンダリインデックスの使用。テーブルの形は GSI のドキュメントの GameScores の例により、値は Claude が作った

容量のモードと RCU・WCU

モード	しくみ	向く場面
オンデマンド	容量を計画しない。前に到達したトラフィックまで即座に受け入れ、使った分だけ払う。ほとんどの場合の既定	予測できない・新しいアプリ・急なピーク
プロビジョンド	1 秒あたりの読み書き（RCU・WCU）を指定し、時間単位で払う。DynamoDB Auto Scaling で自動調整	安定していて予測できる・コストを予測したい

RCU：4 KB までを 1 秒に 1 回（強力な整合性）

3 KB の項目を 1 秒に 80 回読む

3 ÷ 4 → 切り上げて 1 × 80 = 80 RCU

結果整合性なら半分の 40 RCU

WCU：1 KB までを 1 秒に 1 回の書き込み

512 バイトの項目を 1 秒に 100 回書く

0.5 → 切り上げて 1 × 100 = 100 WCU

項目のサイズは単位ごとに切り上げる

切り替え：プロビジョンドからオンデマンドへは 24 時間に最大 4 回、オンデマンドからプロビジョンドへはいつでも。どちらも 1 桁ミリ秒のレイテンシーと同じ SLA。

出典：DynamoDB のスループットキャパシティ（「オンデマンドモードは、ほとんどの DynamoDB ワークロードで推奨されるデフォルトのスループットオプションです。」）／DynamoDB プロビジョンドキャパシティモード（「1 つの書き込みキャパシティユニット (WCU) は、最大サイズが 1 KB の項目の場合、1 秒あたり 1 回の書き込みを表します。」）／DynamoDB オンデマンドキャパシティモード（「24 時間のローリングウィンドウで、テーブルをプロビジョンドキャパシティモードからオンデマンドモードに最大 4 回切り替えることができます。」）

整合性・トランザクション・テーブルクラス

機能	内容	ひっかけ
結果整合性の読み込み	既定。直前の書き込みが反映されていないことがある。強力な整合性の半分のコスト	DAX がキャッシュするのはこちら
強力な整合性の読み込み	ConsistentRead=true。成功した以前のすべての書き込みを反映	テーブルと LSI だけ。GSI・Streams は不可
トランザクション	TransactWriteItems は最大 100 の書き込みをオールオアナッシングで。同じアカウント・リージョンの複数のテーブルにまたがれる	「両方成功か両方なし」の合図
DynamoDB Auto Scaling	Application Auto Scaling で、プロビジョンドの容量をトラフィックに合わせて上下させる	オンデマンドとは別。プロビジョンドのまま
テーブルクラス 標準-IA	ストレージが主なコストのテーブル向け。注文履歴・古い投稿・ログ。あとから変えられる	データを消す TTL とは別。データは残る

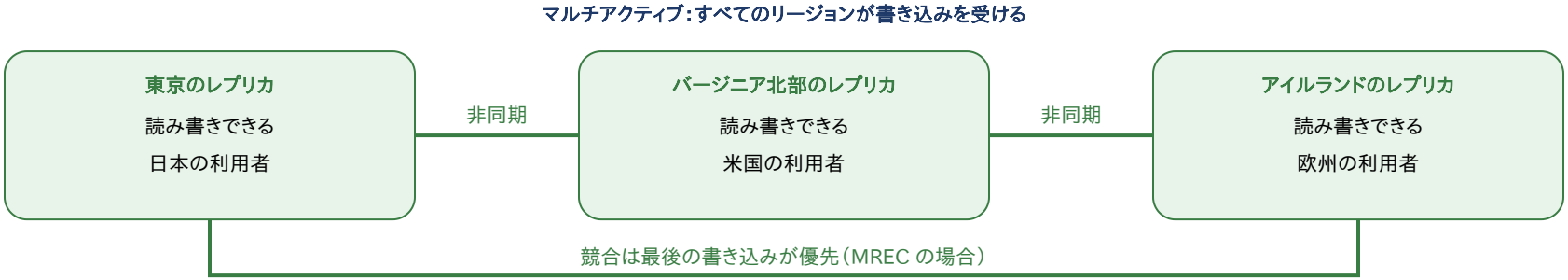
出典：DynamoDB の読み取り整合性（「結果整合性のある読み込みは、強力な整合性のある読み込みの半分のコストで済みます。」「グローバルセカンダリインデックスまたは DynamoDB ストリームからの強力な整合性のある読み込みはサポートされていません。」）／Amazon DynamoDB Transactions: 仕組み／DynamoDB Auto Scaling によるスループットキャパシティの自動管理／DynamoDB テーブルクラス（「DynamoDB Standard-Infrequent Access (DynamoDB 標準-IA) テーブルクラスは、ストレージが主要なコストとなるテーブル用に最適化されています。」）

第2章

DynamoDB の機能と保護

グローバルテーブル、DynamoDB Streams と Kinesis Data Streams、TTL、ポイントインタイムリカバリとオンデマンドバックアップ、S3 へのエクスポート、暗号化、DAX。

グローバルテーブル：どのリージョンでも書ける

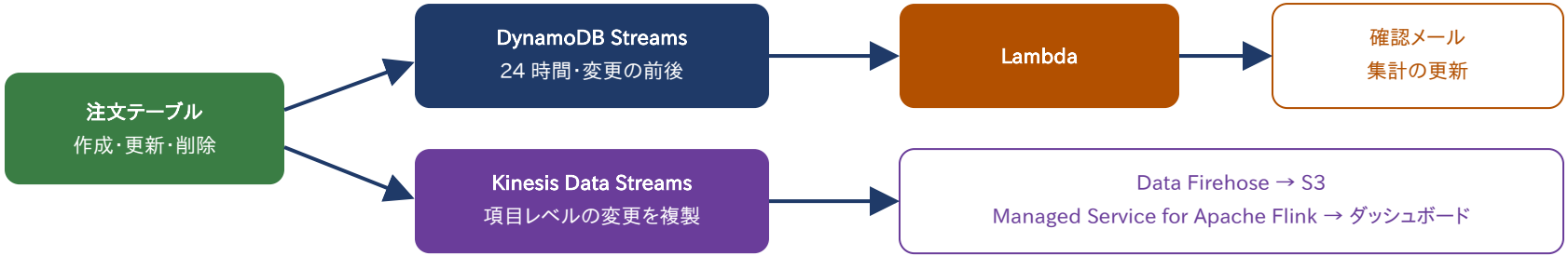


Aurora Global Database との違い：Aurora は書き込みが 1 つのプライマリリージョンだけ。DynamoDB のグローバルテーブルはすべてのレプリカで書ける。複製の仕組みを自分で作る必要がない。

整合性モード：既定はマルチリージョンの結果整合性（MREC）。マルチリージョンの強力な整合性（MRSC）も選べる。作成後に変えられず、MRSC は同じアカウントだけ。

出典：グローバルテーブル - マルチアクティブ、マルチリージョンレプリケーション（「グローバルテーブルは、マルチリージョンの結果整合性 (MREC) およびマルチリージョンの強力な整合性 (MRSC) の 2 つの整合性モードをサポートしています。」「同一アカウントモデルとマルチアカウントモデルの両方で、マルチリージョン書き込み、非同期レプリケーション、最後の書き込み優先の競合解決、および同じ請求モデルがサポートされます。」）。リージョンの組み合わせは Claude が作った例

変更を流す・古い項目を消す



機能	内容	選ぶ合図
DynamoDB Streams	項目レベルの変更を時間順に最大 24 時間。各レコードは 1 回だけ、変更と同じ順	「書き込まれたら Lambda で処理」
Kinesis Data Streams	変更を Kinesis Data Streams に複製し、Firehose や Flink につなぐ	「Firehose・Flink で分析・保存」
TTL	項目ごとの有効期限（秒単位の UNIX エポック時間）を過ぎたら自動で削除。書き込みスループットを使わない。削除は数日以内	「セッションを 24 時間で消す」「消すのに WCU を使わない」

出典：DynamoDB Streams の変更データキャプチャ（「DynamoDB Streams は、DynamoDB テーブル内の項目レベルの変更に関するシーケンスを時間順にキャプチャし、その情報を最大 24 時間ログに保存します。」）／Kinesis Data Streams を使用して DynamoDB への変更をキャプチャする／DynamoDB での Time to Live (TTL) の使用（「削除は、通常は有効期限が切れてから数日以内に行われます。」）

バックアップ・エクスポート・暗号化

機能	内容	選ぶ合図
ポイントインタイムリカバリ（PITR）	継続的に自動バックアップ。1〜35 日（既定 35）の任意の秒に復元。最新は通常、現在の 5 分前	「誤った書き込みを数分前に戻す」
オンデマンドバックアップ	テーブル全体の完全なバックアップ。作った時点の状態に戻せる。本番に影響しない	「何年も残す」「規制への対応」
S3 へのエクスポート	PITR の期間の任意の時点を S3 に。非同期で RCU を使わない。全体と増分。PITR の有効化が必要	「Athena で分析」「本番に影響させない」
保管時の暗号化	すべてのユーザーデータが常に KMS で暗号化。キーの種類（AWS 所有・AWS マネージド・カスタマーマネージド）はいつでも変更できる	「既定で暗号化されているか」→ される

RDS との違い：RDS の PITR も最長 35 日だが、DynamoDB では保持期間を変えても PITR の料金は変わらない（テーブルとインデックスのサイズで決まる）。

Streams は復元に使わない：変更の記録は 24 時間しか残らず、テーブルを戻す機能ではない。グローバルテーブルのレプリカも、誤った上書きまで複製される。

出典：DynamoDB でポイントインタイムリカバリを有効にする（「復旧期間を (35 日から 1 日などに) 変更しても、料金は削減されません。」）／DynamoDB のバックアップと復元（「オンデマンドバックアップを使用すると、DynamoDB テーブル全体を、バックアップ作成時の正確な状態に復元できます。」）／Amazon S3 への DynamoDB データのエクスポート: 仕組み（「エクスポート機能を使用するには、テーブルで PITR を有効にする必要があります。」）／保管時の DynamoDB 暗号化（「キーの種類は、いつでも切り替えることができます。」）

DAX：DynamoDB の前のインメモリキャッシュ



向く：マイクロ秒の応答が要る。同じキーを何度も読む。読み込みが多い・急に増えて RCU を節約したい。DynamoDB と API が互換なので、アプリの変更は最小限。

向かない：強い整合性の読み込みが必要。書き込みが多い（ノード間の複製が増える）。同じ読み込みが少ない（キャッシュヒット率が 90% を超えると最もよく働く）。

出典：DynamoDB Accelerator (DAX) とインメモリアクセラレーション（「DAX は、DynamoDB と互換性のあるキャッシュサービスで、条件の厳しいアプリケーションで高速なインメモリパフォーマンスを可能にします。」「DAX は、キャッシュヒットレートが 90% を超える場合に最適に動作します。」）／DAX: 仕組み（「Amazon DynamoDB Accelerator (DAX) は、Amazon Virtual Private Cloud (Amazon VPC) 環境内で実行するように設計されています。」）

第3章

ElastiCache とキャッシュ戦略

サーバーレスとノードベース、Valkey・Redis OSS と Memcached、レプリケーションとマルチ AZ、グローバルデータストア、認証、遅延読み込み・書き込みスルー・TTL。

エンジンと形を選ぶ

機能	Memcached	Valkey・Redis OSS
データ型	シンプル	複雑（ソート済みセットなど）
マルチスレッド	できる	できない
複製・高可用性	なし	できる
自動フェイルオーバー	なし	できる
バックアップ	サーバーレスだけ	できる
Pub/Sub	なし	できる

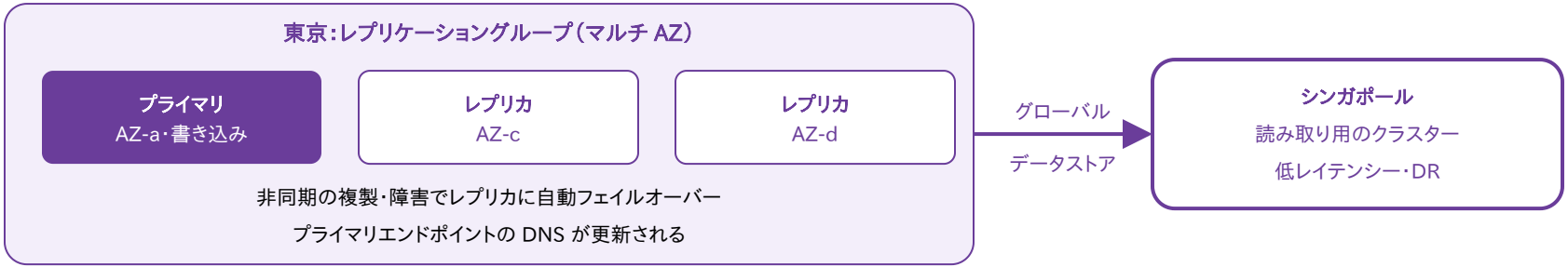
Memcached を選ぶ：できるだけシンプル、複数のコアを持つ大きなノード、ノードの追加・削除でスケールアウト・イン、オブジェクトのキャッシュ。失っても DB から作り直せるデータ。

Valkey・Redis OSS を選ぶ：複製と自動フェイルオーバー、バックアップ、Pub/Sub、順位表（ソート済みセット）、セッション。

サーバーレス：ノードを設定せず 1 分未満で作れ、自動で垂直・水平にスケール。データは複数の AZ に複製され、99.99% の可用性 SLA。

出典：Valkey、Memcached、および Redis OSS のノードベースのクラスターの比較（「複数のコアまたはスレッドを持つ大きなノードを実行する必要がある。」）／Amazon ElastiCache とは（「ElastiCache サーバーレスでは、可用性の高いキャッシュを 1 分未満で作成でき、インスタンスをプロビジョニングしたり、ノードやクラスターを設定したりする必要がありません。」）／ElastiCache の一般的なユースケース（「Valkey または Redis OSS のソート済みセットを使用すると、リーダーボードの計算の複雑性を、アプリケーションからクラスターに移すことができます。」）。表は英語の比較表を Claude が訳した

可用性・リージョン間・認証

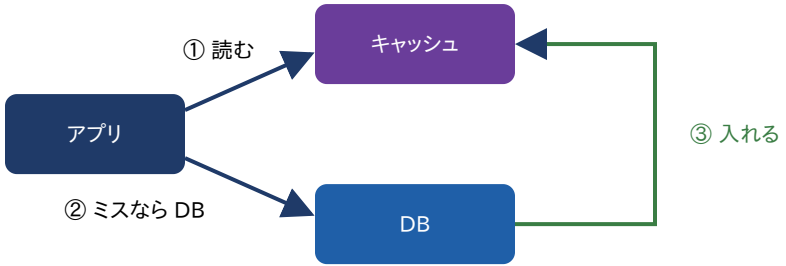


機能	内容	ひっかけ
マルチ AZ	プライマリの障害でレプリカに自動で切り替える。単一ノードは止まると全データを失う	非同期なので少し失うことがある
グローバルデータストア	Valkey・Redis OSS のクラスターをリージョン間で複製。ノードベースのクラスターだけ	DynamoDB のグローバルテーブルとは別
RBAC（旧 AUTH）	ユーザーごとの認証。Secrets Manager でパスワードを自動ローテーション。サーバーレスでは必須	セキュリティグループや暗号化とは別の層

出典：レプリケーショングループを使用する高可用性（「クラスターが何らかの理由で停止すると、クラスターのすべてのデータが失われます。」）／グローバルデータストアを使用した AWS リージョン間のレプリケーション（「グローバルデータストアは現在、ノードベースのクラスターでのみ使用できます。」）／Valkey および Redis OSS AUTH コマンドによる認証（「すべてのサーバーレスキャッシュでは、認証時に RBAC を使用する必要があります。」）

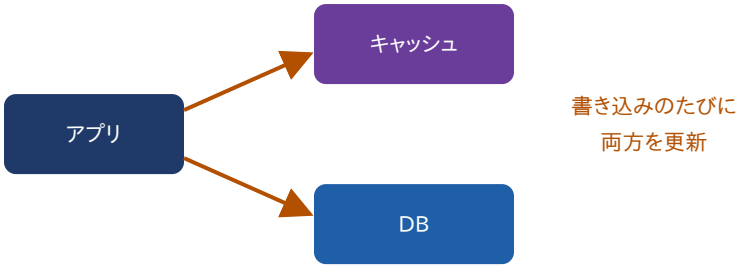
キャッシュ戦略：遅延読み込みと書き込みスルー

遅延読み込み（ミスしたときだけ入れる）



要求されたデータだけ。ノードが空になっても動く。ミスのとき 3 回の往復。DB が変わってもキャッシュは古いまま。

書き込みスルー（DB に書くたびに入れる）



古くならない。読まれないデータまで入ってむだ。新しいノードは DB が更新されるまで空。

TTL を足す：それぞれの書き込みに有効期限（TTL）を付けて、遅延読み込みの「古いデータ」と書き込みスルーの「むだなデータ」を抑える。DynamoDB の TTL（項目の削除）とは別のもの。

出典：Memcached のキャッシュ戦略（「必要なときにのみキャッシュにデータを読み込むキャッシュ戦略です。」「書き込みスルー戦略では、データがデータベースに書き込まれると常にデータを追加するか、キャッシュのデータを更新します。」「有効期限 (TTL) は、キーの有効期限までの秒数を指定する整数値です。」）

まとめ

DynamoDB の基本
PK・SK、GSI はあとから・結果整合性だけ、LSI は作成時・強い整合性も。オンデマンドが既定、安定ならプロビジョンド

DynamoDB の機能
グローバルテーブル=マルチアクティブ、Streams=24 時間、TTL=WCU なしで削除、PITR=35 日、エクスポート=RCU なし

DAX
DynamoDB 専用・マイクロ秒・結果整合性の読み込みだけキャッシュ・コード変更は最小限

ElastiCache
Valkey・Redis OSS=複製・順位表、Memcached=シンプル・マルチスレッド、遅延読み込みと書き込みスルーと TTL

ひっかけ	正しくは
運用中のテーブルに LSI を追加	GSI
GSI で強力な整合性の読み込み	テーブルか LSI
PITR で 7 年残す	オンデマンドバックアップ
DynamoDB を複数リージョンで書くのに Aurora Global	グローバルテーブル
書き込みが多く強い整合性に DAX	DAX は向かない
複製と自動フェイルオーバーに Memcached	Valkey・Redis OSS

次の #17 では、VPC の基本を学ぶ。

出典:このPDFの各ページの資料